



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

* Chess has an astronomically large number of possible game states (estimated at around 10^{40} positions). To pre-compute and store every possible percept sequence and action would require more memory than exists in the universe.

* A table-driven agent can't reason or adapt. It only looks up exact matches. If a percept sequence hasn't been seen before, the agent has no entry to consult.

* Even if you could somehow build the table for a short game, as soon as the environment grows in complexity, the table becomes unmanageable.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In `SimpleReflexAgent` code, the "Condition-Action Rules" from Lecture 02 are represented by the if / elif statements inside the `sense_and_act` method. Each one is a direct translation of the theoretical idea: "IF condition THEN action."

Here's the breakdown:

```
def sense_and_act(self, percept: dict) -> str:
    # Rule 1: If food is here, collect it
    if percept.get('food_here'):
        return 'Suck' # Condition: food_here --> Action: Suck

    # Rule 2: If wall ahead, turn left
    elif percept.get('wall_ahead'):
        return 'Left' # Condition: wall_ahead --> Action: Left

    # Rule 3: If toxin here, move away (basic avoidance)
    elif percept.get('toxin_here'):
        return 'Right' # Condition: toxin_here --> Action: Right

    # Rule 4: Default action: move forward
    else:
        return 'Up' # Condition: none of the above --> Action: Up
```

How these lines map to Lecture 02

Condition: Each if or elif checks a percept (e.g. , `food_here`, `wall_ahead`, `toxin_here`).

Action: Each return specifies the agent's response (e.g., `Suck`, `Left`, `Right`, `Up`).

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

- * In partially Observability, agent only perceives local booleans like wall_ahead, food_here, toxin_here and it does not see the full map or know the global layout of the environment.
- * Also the agent has no memory of the past states or actions that it has made and does not remember that it already turned left three times in a row.
- * So without history, it cannot detect that it's repeating the same cycle.
- * Every time the agent sees a wall ahead, it turns left.
In a corner or U-shaped wall, the percept sequence repeats. Since the agent doesn't remember that it has been here before, it keeps applying the same rule again and again.
- * In Table-Driven Agents, it fails because of combinatorial explosion and Simple Reflex Agents fail because of partial observability and no history.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In code, the transition model is represented by how the agent updates its internal state when the environment changes.

```
self.visited_cells.add(agent_pos)
```

Each time the agent moves, it records the new position in visited_cells.

This models the idea that “the world evolves” as the agent takes actions cells that were once unvisited become visited.

By checking if left_cell in self.visited_cells (and similar conditions), the agent uses its memory of past transitions to avoid revisiting the same states unnecessarily.

The transition model here is simplified: it doesn’t simulate opponent movement or food disappearing, but it does capture the agent’s own progression through the grid.

Sensor Model

The sensor model is represented by how the agent interprets the percept dictionary:

```
if percept.get('food_here'):
    return 'Suck'
```

This is the agent’s direct mapping from sensors to actions: if the sensor says “food_here,” the agent acts accordingly.

Also compute neighbor validity using percept flags:

```
is_left_valid = (x - 1 >= 0) and not percept.get('wall_left')
is_right_valid = (x + 1 < self.width) and not percept.get('wall_right')
```

This is the sensor model in action: the agent uses perceptual input (wall_left, wall_right, etc.) to decide which moves are possible.

It then combines this with memory (visited_cells) to choose between valid moves.



Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING